

Package ‘LDZipMatrix’

July 28, 2026

Type Package
Title Efficient Compression and Random Access for Linkage Disequilibrium Matrices
Version 1.0
Date 2025-08-15
Author Sayantan Das
Maintainer Sayantan Das <sayantand@23andme.com>
Description LDZipMatrix provides tools to compress, store, and access large-scale linkage disequilibrium (LD) matrices efficiently. It supports reading and writing compressed LD data with minimal storage overhead, while enabling fast random access from R through a C++ backend.
License file LICENSE.txt
Imports Rcpp (>= 1.0.8), RSQLite, DBI
LinkingTo Rcpp
RoxygenNote 7.3.3
Encoding UTF-8

Contents

buildIndex	1
fetchLD	2
fetchVariants	5
getNeighbors	6
LDZipMatrix-class	7
LDZipMatrix-metadata	9
Index	12

buildIndex	<i>Build an index for querying by variant ID from an LDZipMatrix object</i>
------------	---

Description

This function builds an index on variant identifiers from the associated `.vars.txt` file of the `LDZipMatrix` object. Optionally one can pass their own variant file.

Usage

```
buildIndex(ld, variant_file)
```

Arguments

ld An external pointer to an LDZipMatrix object, typically created with LDZipMatrix().

variant_file optional path to the variant annotation file (e.g. a .pvar file) containing the variant information (CHROM, POS, ID, REF, ALT). A standard PLINK2 .pvar file works here.

Details

The index allows efficient mapping from variant IDs to internal integer indices, which can then be used with fetchLD() or getNeighbors().

Value

NULL. The index is written to the SQLite database (\$prefix.sqlite)

Examples

```
## Not run:
# Open an LDZip matrix
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")
ld <- LDZipMatrix(prefix)

# Build an index
idx <- buildIndex(ld)

# Fetch LD values by variant IDs
fetchLD(ld, "rs587755077", c("rs587631919", "rs587661542"), type = "PHASED_R")

## End(Not run)
```

fetchLD

Fetch LD values from an LDZipMatrix object

Description

This function retrieves linkage disequilibrium (LD) values from a compressed LDZipMatrix object. Queries can be made either by integer indices (1-based) or by variant identifiers (IDs). If both indices and variant IDs are provided, variant IDs take priority.

Usage

```
fetchLD(
  ld,
  row,
  col,
  types = c("UNPHASED_R"),
  pairwise = FALSE,
  simplify = TRUE
)
```

Arguments

<code>ld</code>	An external pointer to an LDZipMatrix object, typically created with <code>LDZipMatrix()</code> .
<code>row, col</code>	An Integer (1-based), character scalar/vector of variant indices/IDs, or genomic region string (e.g., "chr1:10000-20000", "22:16050000-16051000"). Genomic regions are inclusive on both start and end positions.
<code>types</code>	a character scalar/vector of types of linkage statistics to return. Choices are "PHASED_R", "UNPHASED_R", "PHASED_R2", "UNPHASED_R2", "D", "DPRIME".
<code>pairwise</code>	Logical flag controlling the return type: • TRUE: return pairwise LD values, where each element of row is matched with the corresponding element of col (both must have equal length). • FALSE: return the full LD submatrix spanning all combinations of row and col.
<code>simplify</code>	Logical flag. If TRUE, the return value is simplified according to the shape of the query and the number of statistics: • Single statistic, (1 x 1), (1 x N) or (N x 1): a vector. • Single statistic, (N x N): a matrix. • Multiple statistics, (1 x 1), (1 x N) or (N x 1): a data.frame. • Multiple statistics, (N x N): a list of matrix. If FALSE, results are always returned as a list of matrices, one per statistic. Note: This parameter is ignored when <code>pairwise=TRUE</code>.

Value

When `pairwise=FALSE` (default):

- A numeric vector if either row or col is a vector (the other scalar), and only one statistic is requested.
- A numeric matrix if both row and col are vectors and a single statistic is requested.
- A data.frame if either row or col is a vector (the other scalar), and multiple statistics are requested.
- A list of matrices if both row and col are vectors and multiple statistics are requested (or if `simplify=FALSE`)

When `pairwise=TRUE`:

- Always returns a data.frame with one column per requested statistic type, one row per pair, and additional columns `var1` and `var2` to identify the pairs. Results are returned in the same order as the input. Duplicate pairs are allowed.

Note

- In order to use variant identifiers or genomic regions (instead of indices), please index the LDZipMatrix object using `buildIndex(ld)`
- When `pairwise=FALSE`: Variants in the returned object are sorted in order of their variant indices, and not the same order as the input in row or col
- When `pairwise=TRUE`: Results are returned in the same order as the input. Duplicate pairs are allowed.

Examples

```
## Not run:
# Open an LDZip matrix
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")
ld <- LDZipMatrix(prefix)

# 1 x 1 query, single statistic → scalar
fetchLD(ld, 2, 191, types = "PHASED_R")

# 1 x 1 query, multiple statistics → data.frame
fetchLD(ld, 2, 191, types = c("PHASED_R", "DPRIME"))

# 1 x N query, single statistic → vector
fetchLD(ld, 2, c(2,129,191,307), types = "PHASED_R")

# 1 x N query, multiple statistics → data.frame
fetchLD(ld, 2, c(2,129,191,307), types = c("PHASED_R", "DPRIME"))

# N x N query, single statistic → matrix
fetchLD(ld, 1:5, 1:5, types = "PHASED_R")

# N x N query, multiple statistics → list of matrices
fetchLD(ld, 1:5, 1:5, types = c("PHASED_R", "DPRIME"))

## Query with variant identifiers
# Build index (if it doesn't exist already)
buildIndex(ld)

# 1 x 1 query, single statistic → scalar
fetchLD(ld, "rs587755077", "rs587631919", types = "PHASED_R")

# Define a small set of variant IDs
vars <- c("rs587725733", "rs587631919", "rs587661542", "rs587718290", "rs587729333")

# 1 x N query, single statistic → vector
fetchLD(ld, "rs587755077", vars, types = "PHASED_R")

# 1 x N query, multiple statistics → data.frame
fetchLD(ld, "rs587755077", vars, types = c("PHASED_R", "DPRIME"))

# N x N query, single statistic → matrix
fetchLD(ld, vars, vars, types = "PHASED_R")

# N x N query, multiple statistics → list of matrices
fetchLD(ld, vars, vars, types = c("PHASED_R", "DPRIME"))

## Region-based queries
# Same region LD matrix
fetchLD(ld, "22:16050000-16051000", "22:16050000-16051000")

## Pairwise queries (always returns data.frame)
# Fetch LD for specific pairs (row[i] with col[i])
rows <- c(1, 5, 10)
cols <- c(2, 8, 15)
fetchLD(ld, rows, cols, types = "PHASED_R", pairwise = TRUE)
```

```
# Pairwise with variant IDs
row_vars <- c("rs587725733", "rs587631919", "rs587661542")
col_vars <- c("rs587755077", "rs587718290", "rs587729333")
fetchLD(ld, row_vars, col_vars, types = "PHASED_R", pairwise = TRUE)

# Pairwise with multiple statistics (returns data.frame with multiple columns)
fetchLD(ld, rows, cols, types = c("PHASED_R", "DPRIME"), pairwise = TRUE)

## End(Not run)
```

fetchVariants

Fetch variant information from an LDZipMatrix object

Description

This function retrieves variant metadata including chromosome, position, variant ID, reference allele, and alternate allele from a compressed LDZipMatrix object. Queries can be made by integer indices (1-based), variant identifiers (IDs), or genomic regions.

Usage

```
fetchVariants(ld, variants)
```

Arguments

<code>ld</code>	An external pointer to an LDZipMatrix object, typically created with <code>LDZipMatrix()</code> .
<code>variants</code>	Integer (1-based), character scalar/vector of variant indices/IDs, or genomic region string (e.g., "chr1:10000-20000", "22:16050000-16051000"). Genomic regions are inclusive on both start and end positions.

Value

A `data.frame` with the following columns:

- `idx`: 1-based variant index
- `CHROM`: chromosome
- `POS`: position (base pairs)
- `ID`: variant identifier (e.g., `rsID`)
- `REF`: reference allele
- `ALT`: alternate allele

Results are sorted by variant index.

Note

- In order to use variant identifiers or genomic regions (instead of indices), please index the LDZipMatrix object using `buildIndex(ld)`
- Variants in the returned `data.frame` are sorted by variant index

See Also

[fetchLD](#), [buildIndex](#)

Examples

```
## Not run:
# Open an LDZip matrix
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")
ld <- LDZipMatrix(prefix)

# Build index (if it doesn't exist already)
buildIndex(ld)

# Query by numeric indices
fetchVariants(ld, 1:10)

# Query by variant IDs
vars <- c("rs587725733", "rs587631919", "rs587661542")
fetchVariants(ld, vars)

# Query by genomic region
fetchVariants(ld, "22:16050000-16051000")

## End(Not run)
```

getNeighbors

Find neighboring variants in an LDZipMatrix

Description

This function finds all variants exceeding a specified LD threshold for a given statistics type, within a given genomic window around a query variant, using an `LDZipMatrix` object.

Usage

```
getNeighbors(
  ld,
  variant,
  type = "UNPHASED_R2",
  abs_threshold = 0.8,
  genomic_length = 1e+05
)
```

Arguments

<code>ld</code>	An external pointer to an <code>LDZipMatrix</code> object, typically created with <code>LDZipMatrix()</code> .
<code>variant</code>	Integer (1-based)/Character scalar referring to the reference variant index/identifier around which neighbors are sought.
<code>type</code>	Character scalar referring to the type of linkage statistic sought.
<code>genomic_length</code>	Integer scalar referring to the window size in base pairs (e.g., 1e6 for +/-1 Mb).

Value

Integer/Character vector of variant indices/identifiers.

- If variant is a variant index (Integer), the function returns an integer vector (1-based) of variant indices
- If variant is a variant identifier (Character), the function returns a character vector of variant identifiers

Note

- In order to use variant identifiers (instead of indices), please index the LDZipMatrix object using `buildIndex(ld)`

Examples

```
## Not run:

# Open an LDZip matrix
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")
ld <- LDZipMatrix(prefix)

# Build index (if it doesn't exist already)
buildIndex(ld)

# Find neighbors with  $r^2 > 0.8$  within  $\pm 500$ kb of rs587755077
getNeighbors(ld, "rs587755077", type="PHASED_R", abs_threshold = sqrt(0.8), genomic_length=500000)

# Find neighbors within  $\pm 100$ Kb of rs587755077 that are in perfect LD ( $D'=1$ )
getNeighbors(ld, "rs587755077", type="DPRIME", abs_threshold = 1, genomic_length=100000)

# Verify their  $D'$  value
res = getNeighbors(ld, "rs587755077", type="DPRIME", abs_threshold = 1.0, genomic_length=100000)
fetchLD(ld, "rs587755077", res, "DPRIME")

## End(Not run)
```

LDZipMatrix-class

LDZipMatrix Class and Constructor

Description

An LDZipMatrix stores pairwise linkage disequilibrium (LD) statistics between genetic variants using a compressed representation on disk. A single LDZipMatrix file can also hold multiple LD statistics (e.g., r , r^2 , D , D'). Instead of recalculating LD on demand, results can be stored once and retrieved whenever necessary. Because LD matrices grow quadratically with the number of variants, they are prohibitively large to hold in dense form. LDZipMatrix stores them in compressed format that minimizes storage while still supporting efficient random access to rows, columns, or individual elements as needed.

Usage

```
LDZipMatrix(prefix)
```

Arguments

`prefix` Character string. The file prefix of the LDZip matrix (without extensions).

Details

The LDZipMatrix class is implemented as an *external pointer* to a C++ object. The underlying C++ object manages access to a set of compressed files on disk that together encode the matrix in *Compressed Sparse Column (CSC)* format. This format stores, for each column, a pointer to its nonzero entries (`.p.bin`), the corresponding row indices (`.i.bin`), and one or more compressed value streams (`.x.<stats>.bin`). The row indices are delta encoded and compressed in chunks using `zstd` compression (`.i.bin`), with chunk boundaries indexed in (`.i.bin.index`) for efficient random access

Each `.x.<stats>.bin` file corresponds to a particular LD statistic (e.g., PHASED_R, UNPHASED_R2, D, DPRIME). Multiple such files may coexist for the same matrix prefix, allowing the same structural representation (`i`, `p`) to be reused across different LD formats.

The full LD matrix is never loaded into memory; instead, only the requested rows, columns, or elements are decompressed on demand. This design enables efficient random access to very large LD matrices without requiring gigabytes of RAM.

Value

An object of class "LDZipMatrix" (an external pointer to a C++ instance).

Note

A valid LDZipMatrix requires several files with the same prefix:

- `<prefix>.i.bin` – delta-encoded row indices for nonzero entries, chunked and compressed with `zstd`
- `<prefix>.i.bin.index` – chunk boundaries for efficient random access to row indices
- `<prefix>.p.bin` – column pointers marking the start of each column in `i` and `x`
- `<prefix>.x.<stats>.bin` – compressed numeric values for a given LD statistic, chunked and compressed with `zstd`
- `<prefix>.x.<stats>.bin.index` – chunk boundaries for efficient random access to each statistic
- `<prefix>.meta.json` – json file describing associated metadata (version, `chunk_size`, `stats`, etc.)
- `<prefix>.vars.txt` – variant identifiers
- (optionally) `<prefix>.sqlite` – SQLite database with indexed variant information

All of these files must be present in the same directory for the constructor `LDZipMatrix(prefix)` to succeed.

See Also

[LDZipMatrix](#), [getLD](#), [getNeighbors](#)

Examples

```
## Not run:
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")

# Create a new LDZipMatrix handle
ld <- LDZipMatrix(prefix)

# Query metadata
dim(ld)
stats(ld)

# Print and summarize
print(ld)
summary(ld)

# Fetch a single entry (on-demand decompression)
val <- fetchLD(ld, 10, 10, types="PHASED_R")

## End(Not run)
```

LDZipMatrix-metadata *Functions for LDZipMatrix metadata*

Description

Retrieve dimensions and metadata from an LDZipMatrix object

These functions provide convenient accessors for structural information and metadata associated with a compressed LDZipMatrix object. They mirror base R generics (e.g. `nrow()`, `ncol()`, `dim()`) where appropriate, and add specialized accessors for sparse matrix properties and LD-specific metadata.

Usage

```
## S3 method for class 'LDZipMatrix'
nrow(x)

## S3 method for class 'LDZipMatrix'
ncol(x)

## S3 method for class 'LDZipMatrix'
dim(x)

## S3 method for class 'LDZipMatrix'
print(x, ...)

## S3 method for class 'LDZipMatrix'
summary(object, ...)

## S3 method for class 'summary.LDZipMatrix'
print(x, ...)
```

```
nnz(x)
```

```
layout(x, ...)
```

```
stats(x)
```

```
bits(x)
```

Arguments

x An external pointer to an LDZipMatrix object, typically created with LDZipMatrix().

Details

- `nrow(x)`: Return the number of rows (variants) in the matrix.
- `ncol(x)`: Return the number of columns (variants) in the matrix.
- `dim(x)`: Return both number of rows and columns as a vector.
- `nnz(x)`: Return the number of non-zero entries stored in the compressed representation.
- `stats(x)`: Return the set of LD statistics available in the object. Possible values include "PHASED_R", "UNPHASED_R", "PHASED_R2", "UNPHASED_R2", "D", "DPRIME".
- `bits(x)`: Return the number of bits used for compression (e.g. 8, 16, 32).
- `layout(x)`: Return the matrix layout used in compression (e.g. "upper", "full").

These accessors are lightweight and do not decompress the full LD matrix. They are intended for quick inspection and for programmatic use within higher-level analysis workflows.

Value

An integer, logical, or character vector depending on the function:

- `nrow()`, `ncol()` return integers.
- `dim()` returns an integer vector of length two.
- `nnz()` returns an integer.
- `stats()` returns a character vector of available LD statistics.
- `bits()` returns an integer.
- `layout()` returns a character string.

See Also

[LDZipMatrix](#)

Examples

```
## Not run:
prefix <- file.path(system.file("extdata", package = "LDZipMatrix"), "g1k.chr22.ldzip")

# Create a new LDZipMatrix handle
ld <- LDZipMatrix(prefix)

# Query metadata
nrow(ld)
```

```
ncol(ld)
dim(ld)
nnz(ld)
stats(ld)
bits(ld)
layout(ld)
print(ld)
summary(ld)

## End(Not run)
```

Index

bits (LDZipMatrix-metadata), [9](#)
buildIndex, [1](#), [6](#)

dim.LDZipMatrix (LDZipMatrix-metadata),
[9](#)

fetchLD, [2](#), [6](#)
fetchVariants, [5](#)

getLD, [8](#)
getNeighbors, [6](#), [8](#)

layout (LDZipMatrix-metadata), [9](#)
LDZipMatrix, [8](#), [10](#)
LDZipMatrix (LDZipMatrix-class), [7](#)
LDZipMatrix-class, [7](#)
LDZipMatrix-metadata, [9](#)

ncol.LDZipMatrix
 (LDZipMatrix-metadata), [9](#)
nnz (LDZipMatrix-metadata), [9](#)
nrow.LDZipMatrix
 (LDZipMatrix-metadata), [9](#)

print.LDZipMatrix
 (LDZipMatrix-metadata), [9](#)
print.summary.LDZipMatrix
 (LDZipMatrix-metadata), [9](#)

stats (LDZipMatrix-metadata), [9](#)
summary.LDZipMatrix
 (LDZipMatrix-metadata), [9](#)